



УНИВЕРСИТЕТ
ЛОБАЧЕВСКОГО

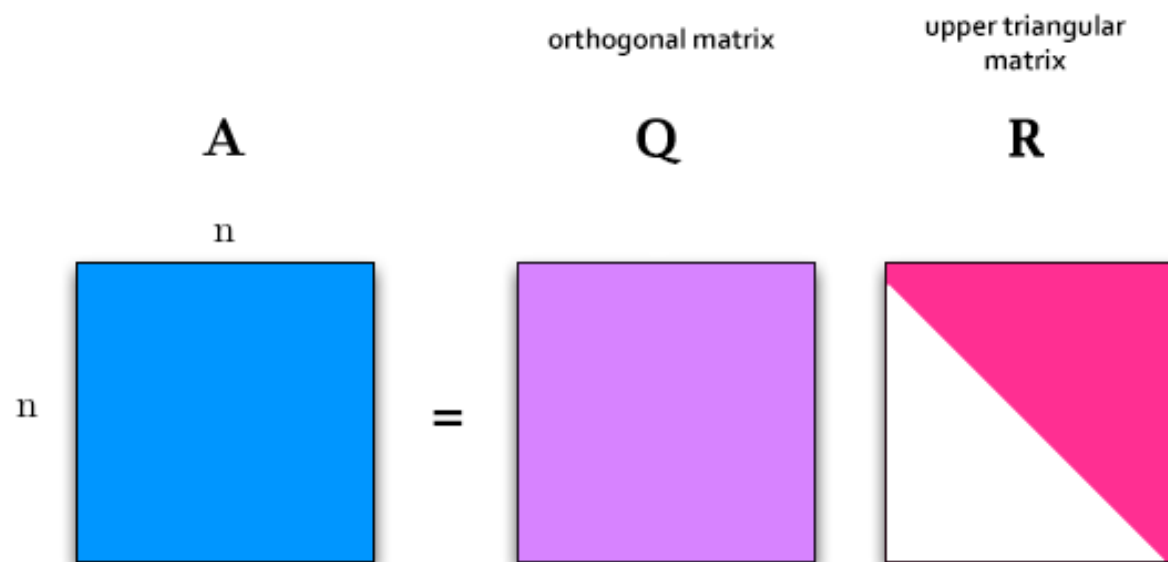
Параллельная реализация QR-разложения

Коробейникова Алиса
Студентка группы 3824М1Пр1
Научный руководитель:
Баркалов К.А., д.т.н., доц.

Описание разложения

QR-разложение матрицы – представление матрицы в виде произведения унитарной (или ортогональной, в вещественном случае) Q ($Q^T Q = E$) и верхнетреугольной матрицы R с положительными диагональными элементами.

Если при этом A невырождена, то QR-разложение единственно и матрица R может быть выбрана так, чтобы её диагональные элементы были положительными вещественными числами.



Применение и получение разложения

QR-разложение является основой одного из методов поиска собственных векторов и чисел матрицы — QR-алгоритма. С помощью разложения можно решить линейную задачу наименьших квадратов.

Также с помощью QR-разложения можно решать СЛАУ $Ax = b$: если известно QR-разложение матрицы, исходная система может быть записана как $QRx=b$. Решением этой задачи является решение треугольной системы $Rx=Q^Tb$. Эта система может быть решена с помощью обратной подстановки, поскольку R – верхняя треугольная матрица.

QR-разложение может быть получено различными методами. Проще всего оно может быть вычислено в процессе ортогонализации Грама-Шмидта. Другие методы QR-разложения основаны на отражениях Хаусхолдера и вращениях Гивенса. Все перечисленные методы имеют ряд ограничений для применения

Постановка задачи

Необходимо реализовать эффективные программные алгоритмы QR-разложения квадратной матрицы A с помощью методов отражений Хаусхолдера и вращений Гивенса, используя различные аппаратные оптимизации, а также применяя, где это возможно, параллелизм. Разработать несколько версий, сравнить их работу друг с другом.

Для разработки использовать язык Си++ и технологии OpenMP для обеспечения параллельного исполнения кода.

В качестве примера приложения QR-разложения на практике выполнить решение задачи QR-разложения хессенберговых матриц, верхней и нижней.

Сравнить работу алгоритма с известными реализациями в сторонних библиотеках.

Проверка корректности

Полученные матрицы Q и R необходимо перемножить и сравнить результат с исходной матрицей A.

Подсчитаем абсолютную ошибку с помощью Нормы Фробениуса (также евклидова норма, соответствующая евклидову пространству матриц относительно фробениусова скалярного произведения, или сферическая норма) представляет собой частный случай p-нормы для $p = 2$:

$$\|A\|_F = \sqrt{\sum_{i=0}^n \sum_{j=0}^m a_{ij}^2}, \text{ где } n, m - \text{размеры матрицы } A.$$

Получим $\|QR - A\|_F$ и $\frac{\|QR - A\|_F}{\|A\|_F}$ для оценки ошибки.

Метод отражений Хаусхолдера

Пусть v – n -мерный ненулевой вектор-столбец. Квадратная матрица H порядка n вида:

$$H = E - \frac{2vv^T}{v^T v}$$

называется отражением Хаусхолдера. Вектор v – вектором Хаусхолдера.

Геометрическая интерпретация умножения отражения Хаусхолдера H на вектор x : вектор Hx получается отражением вектора x относительно гиперплоскости, ортогональной вектору v .

Матрица Хаусхолдера будет симметрична и ортогональна.

В качестве вектора Хаусхолдера v можно взять вектор, вычисляемый по формуле

$v = x \pm \|x\|_2 e_1$, или любой ненулевой, ему коллинеарный.

Применение отражений Хаусхолдера для QR-разложения

Суть алгоритма QR-разложения, реализованного с помощью отражений Хаусхолдера – в последовательном домножении матрицы A слева на отражения Хаусхолдера H с целью получения матрицы R как побочного продукта и Q с помощью явных перемножений в простейшем случае.

На каждом шаге алгоритма k получаем k -ое отражение H_k , домножая на него матрицу A слева получаем $\widetilde{A}_k = H_k H_{k-1} \dots H_1 A$, в которой k первых столбцов имеют нули под главной диагональю:

$$H_1 A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & f_{32} & f_{33} & f_{34} \\ 0 & f_{42} & f_{43} & f_{44} \end{pmatrix}, H_2 H_1 A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & 0 & f_{33} & f_{34} \\ 0 & 0 & f_{43} & f_{44} \end{pmatrix}, H_3 H_2 H_1 A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & 0 & f_{33} & f_{34} \\ 0 & 0 & 0 & f_{44} \end{pmatrix}$$

Выполнив $n-1$ таких шагов, получим верхнюю треугольную матрицу

$R = H_{n-1} \dots H_1 A$. и ортогональную $Q = H_1 H_2 \dots H_{n-1}$.

Описание первой реализации

С помощью ортогональных преобразований приведём матрицу A к треугольному виду.

Чтобы занулить $n-1$ строк матрицы A размером $n \times n$, требуется выполнить $n-1$ шагов. На каждом из них нужно вычислять матрицу отражения $H_k = E - 2w_k w_k^T$ помощью следующих преобразований:

1. $\beta_k = \text{sign}(-\tilde{a}_{kk}) \sqrt{\sum_{i=k}^n \tilde{a}_{ik}^2}$
2. $\mu_k = \frac{1}{\sqrt{2\beta_k^2 - 2\beta_k \tilde{a}_{kk}}}$
3. $w_k = \mu_k (0; \dots; 0; \tilde{a}_{kk} - \beta_k; \tilde{a}_{(k+1)k}; \dots; \tilde{a}_{nk})$, первые $(k-1)$ позиций – нулевые.

На каждом шаге алгоритма домножаем матрицу $\tilde{A}$ с предыдущего шага слева на матрицу отражения и производим следующие вычисления уже над ней.

$$R = H_{n-1} \dots H_1 A = \tilde{A}_n.$$

$$Q = H_1 \dots H_{n-1}.$$

Возможные оптимизации

В связи с выявленными недостатками возможны следующие оптимизации:

- Избавление от матричных умножений: подсчёт $H_k A_k$ и Q по-другому:

$$H_k A_k = (I - 2 v \cdot v^T) A_k = A_k - 2 v (v^T \cdot A_k)$$
$$Q_{n-1} = I$$
$$Q_k = H_k Q_{k-1} = Q_{k-1} - 2 v_k (v_k^T Q_{k-1}), k = n-2, \dots, 0$$
$$Q = Q_0$$

- В связи с этим становится необязательным явно хранить матрицы Хаусхолдера H_k для восстановления матрицы Q , достаточно сохранять вектора длины $(N-k)$ в буфере

Возможные оптимизации

В связи с выявленными недостатками возможны следующие оптимизации:

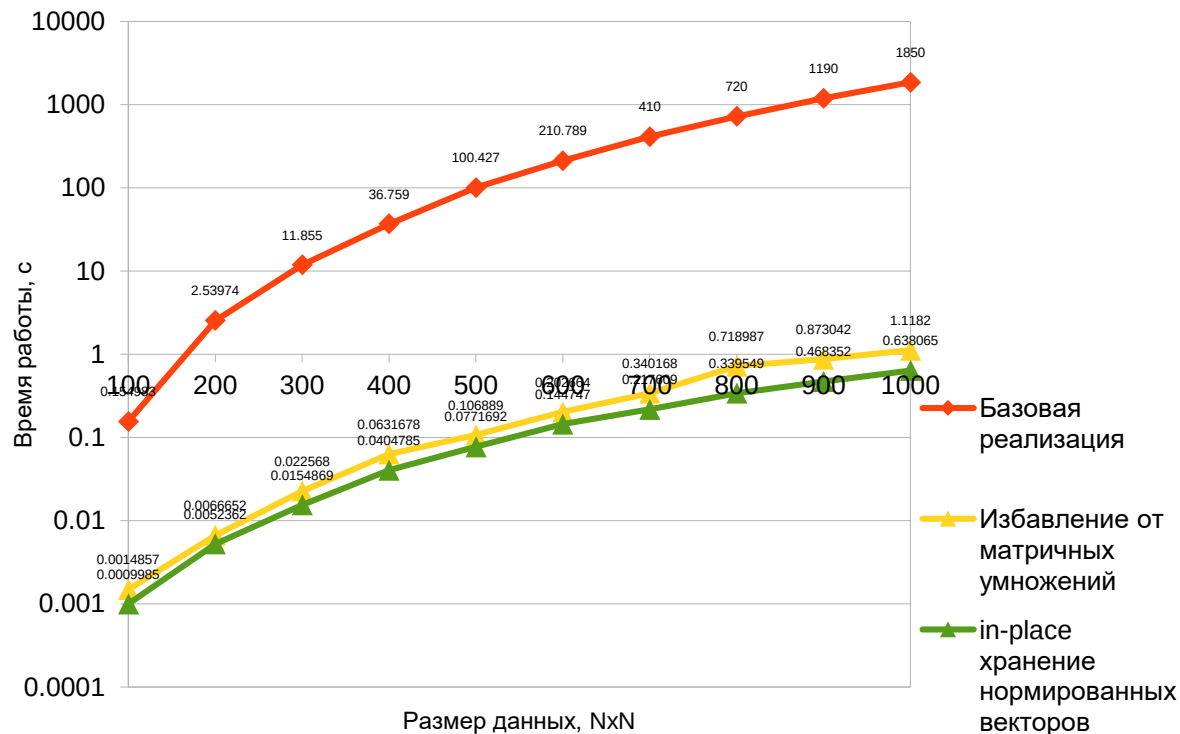
- Работать с нормированными векторами Хаусхолдера v_k (v_{k0} будет равен 1), что сократит их длину, и сохранять их в R под диагональю, без выделения дополнительной памяти:

$$\sigma = -\text{sign}(a_k[0]) \|a_k\| \quad v_k = a_k - \sigma e_k = \left[\underbrace{0; \dots; 0}_{k-1}; a_{kk} - \sigma; a_{k,k+1}; \dots; a_{k,n} \right] \quad \left| H = I - \tau * (v \cdot v^T), \text{ где } \tau = \frac{2}{v^T \cdot v} \right.$$

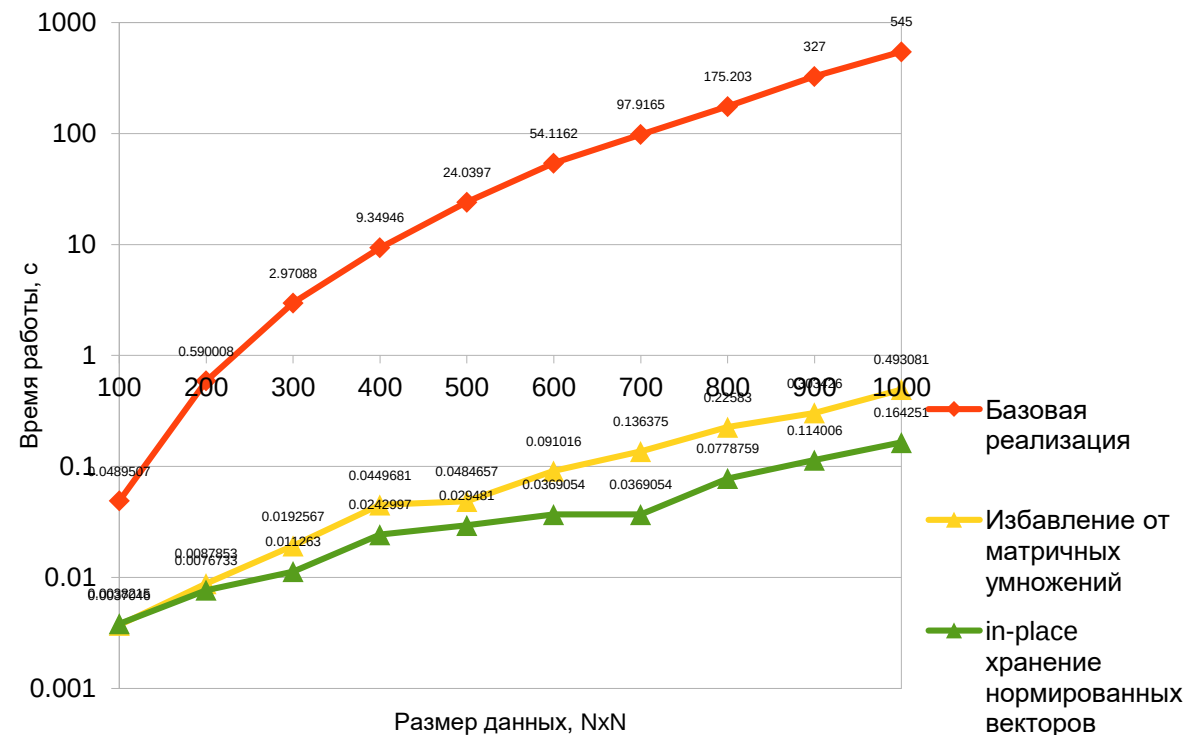
$$H_k A_{k-1} = \begin{bmatrix} \sigma_1 & \widetilde{A}_{12} & \dots & \widetilde{A}_{1k} & \dots & \dots & \widetilde{A}_{1n} \\ v_{11}/v_{10} & \sigma_2 & \dots & \widetilde{A}_{2k} & \dots & \dots & A_{2n} \\ \dots & \dots & \ddots & \dots & \dots & \dots & \dots \\ v_{1k}/v_{10} & \dots & \dots & \sigma_k & \dots & \dots & \widetilde{A}_{kn} \\ v_{1,k+1}/v_{10} & \dots & \dots & v_{k1}/v_{k0} & \widetilde{A}_{k+1,k+1} & \dots & \widetilde{A}_{k+1,n} \\ \dots & \dots & \dots & \vdots & \vdots & \ddots & \dots \\ v_{1n}/v_{10} & \dots & \dots & v_{kn}/v_{k0} & \widetilde{A}_{n,k+1} & \dots & \widetilde{A}_{n,n} \end{bmatrix}$$

Сравнение трёх полученных реализаций

С учётом оптимизаций, ниже представлены замеры однопоточного и параллельного исполнения для трёх программных реализаций:



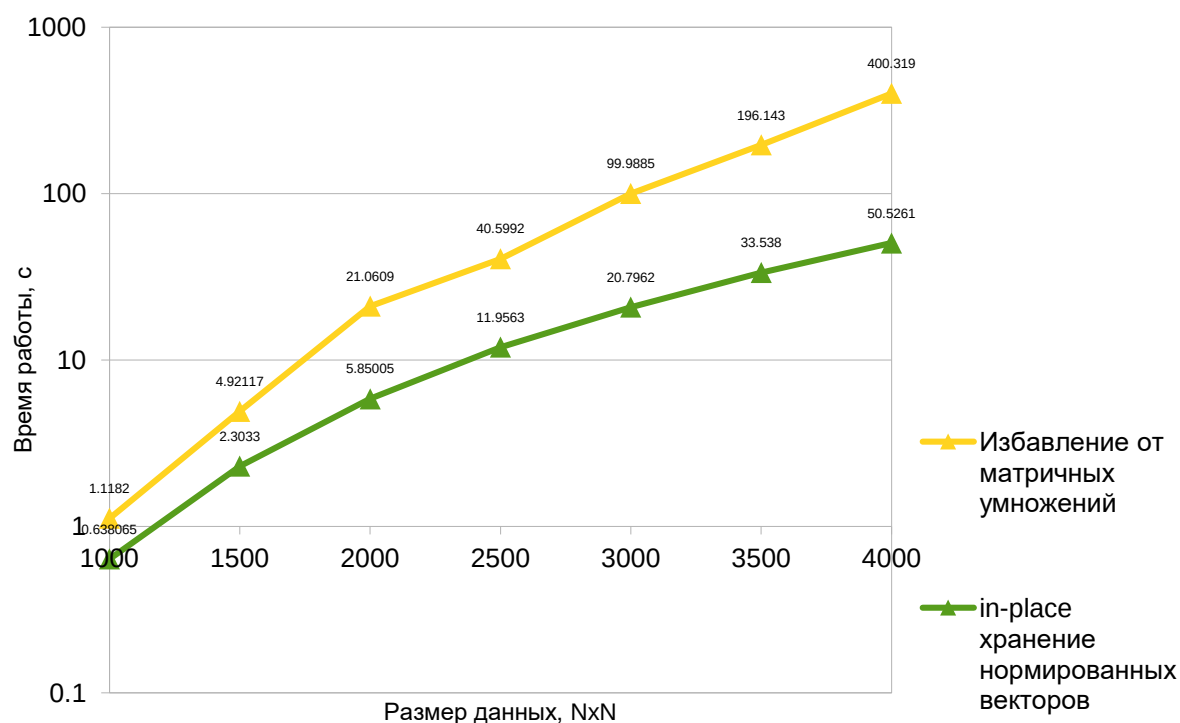
Запуск на одном потоке



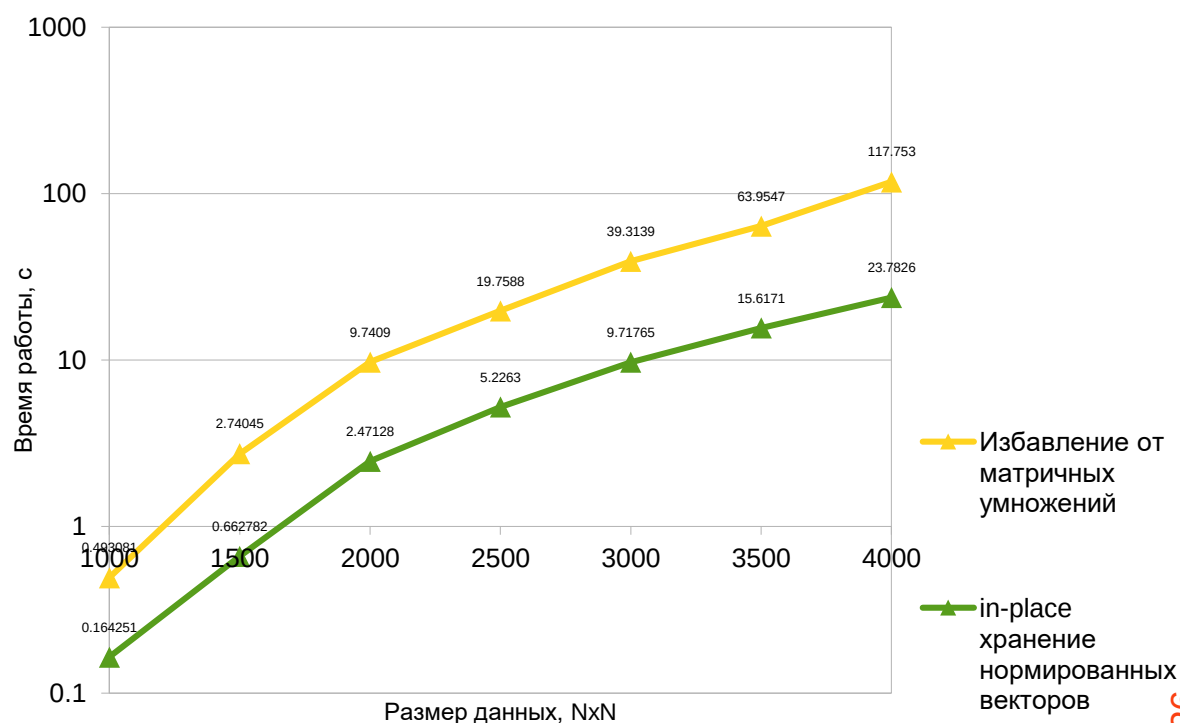
Запуск на восьми потоках

Сравнение трёх полученных реализаций

Благодаря более быстрой работе можно запустить на данных более 1000x1000 элементов:



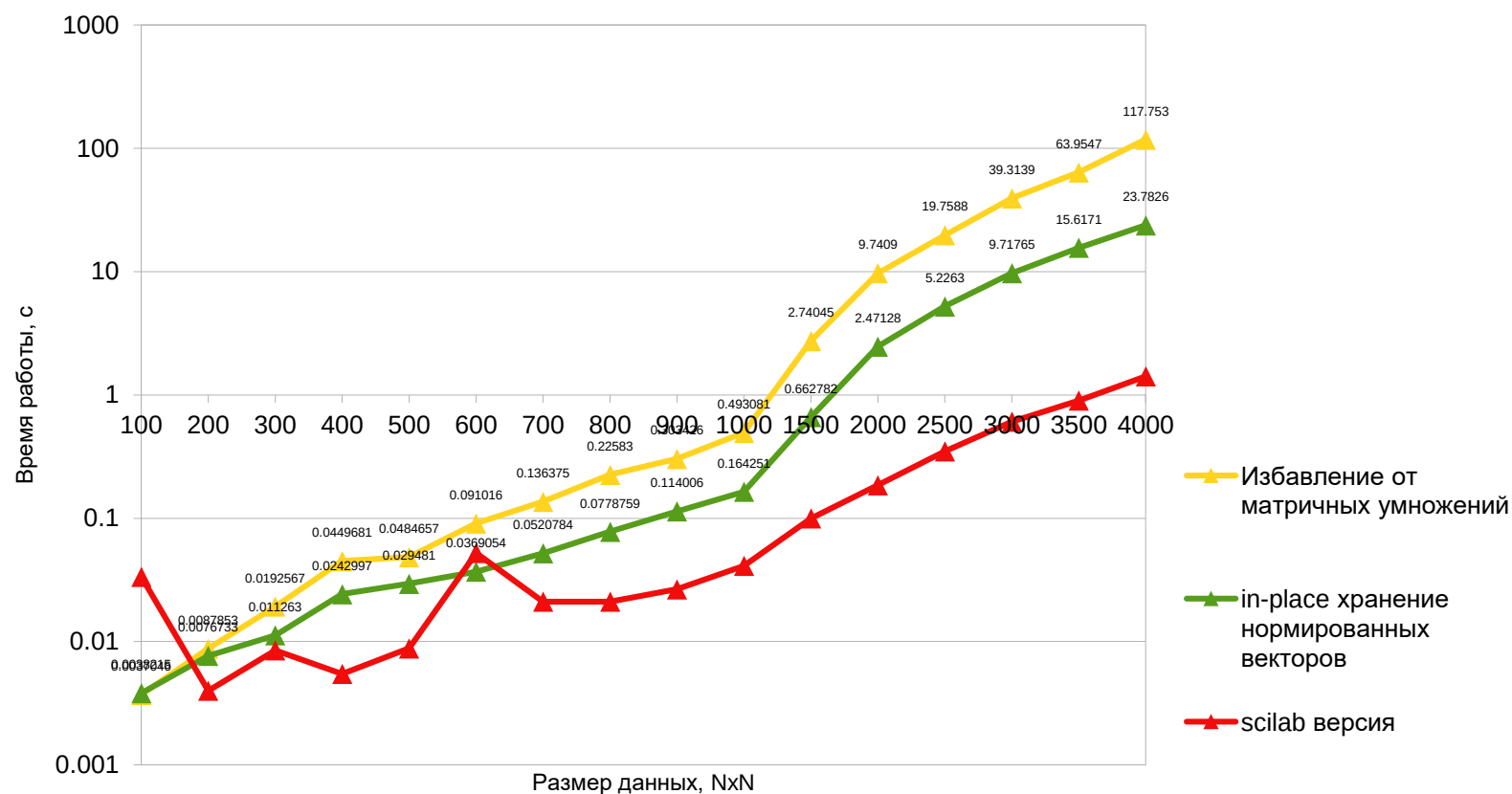
Запуск на одном потоке



Запуск на восьми потоках

Сравнение с библиотекой scilab

Ниже приведено сравнение работы двух оптимальных версий, исполняющихся на восьми потоках, с реализацией QR-разложения в библиотеке scilab:



- Scilab использует LAPACK — библиотеку, написанную на Fortran c:
- Блочными алгоритмами (более оптимальное использование кэша)
- Высокооптимизированными многопоточными BLAS — Basic Linear Algebra Subprograms (Intel MKL/OpenBLAS)
- Ручной векторизацией и ассемблерными вставками

Метод вращений Гивенса

Поворот Гивенса — линейный оператор поворота вектора на некоторый заданный угол.

Матрица вращения (Гивенса) G задаётся позициями столбцов i и j и параметрами $\cos$ и $\sin$ вращения:

$$G_{ij} = \begin{bmatrix} 1 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \cdots & \ddots & / & / & / & / & / & / & / & / & 0 \\ 0 & \cdots & 1 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & \cdots & 0 & c & 0 & \cdots & 0 & -s & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & 1 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ / & / & / & / & / & \ddots & / & / & / & / & / \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 1 & 0 & 0 & \cdots & 0 \\ 0 & \cdots & 0 & s & 0 & \cdots & 0 & c & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 1 & \cdots & 0 \\ / & / & / & / & / & / & / & / & / & \ddots & / \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 1 \end{bmatrix}$$

Это преобразование используется для преобразования матриц или векторов с целью приведения их к определённому виду.

В данном случае, параметр такого преобразования будет подбираться для двух строк матрицы так, чтобы один из поддиагональных элементов преобразуемой матрицы стал нулевым.

Применение вращений Гивенса для QR-разложения

Суть алгоритма QR-разложения, реализованного с помощью вращений Гивенса – в последовательном занулении всех поддиагональных элементов, с целью получения верхнетреугольной матрицы R как побочного продукта и Q с помощью явных перемножений матриц вращения в простейшем случае.

На каждом шаге алгоритма для зануления элемента с индексами (i,j) получаем вращение G_{ji} , домножая на него матрицу A слева получаем , в которой элементы под главной диагональю, левее (i,j), занулены:

$$G_{12}A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ f_{13} & f_{32} & f_{33} & f_{34} \\ f_{14} & f_{42} & f_{43} & f_{44} \end{pmatrix} \quad G_{13}G_{12}A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & f_{32} & f_{33} & f_{34} \\ f_{14} & f_{42} & f_{43} & f_{44} \end{pmatrix} \quad G_{14}G_{13}G_{12}A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & f_{32} & f_{33} & f_{34} \\ 0 & f_{42} & f_{43} & f_{44} \end{pmatrix}$$

$$R = G_{n-1,n} G_{n-2,n} \dots G_{2n} G_{1n} \dots G_{24} G_{23} \dots G_{13} G_{12} A$$

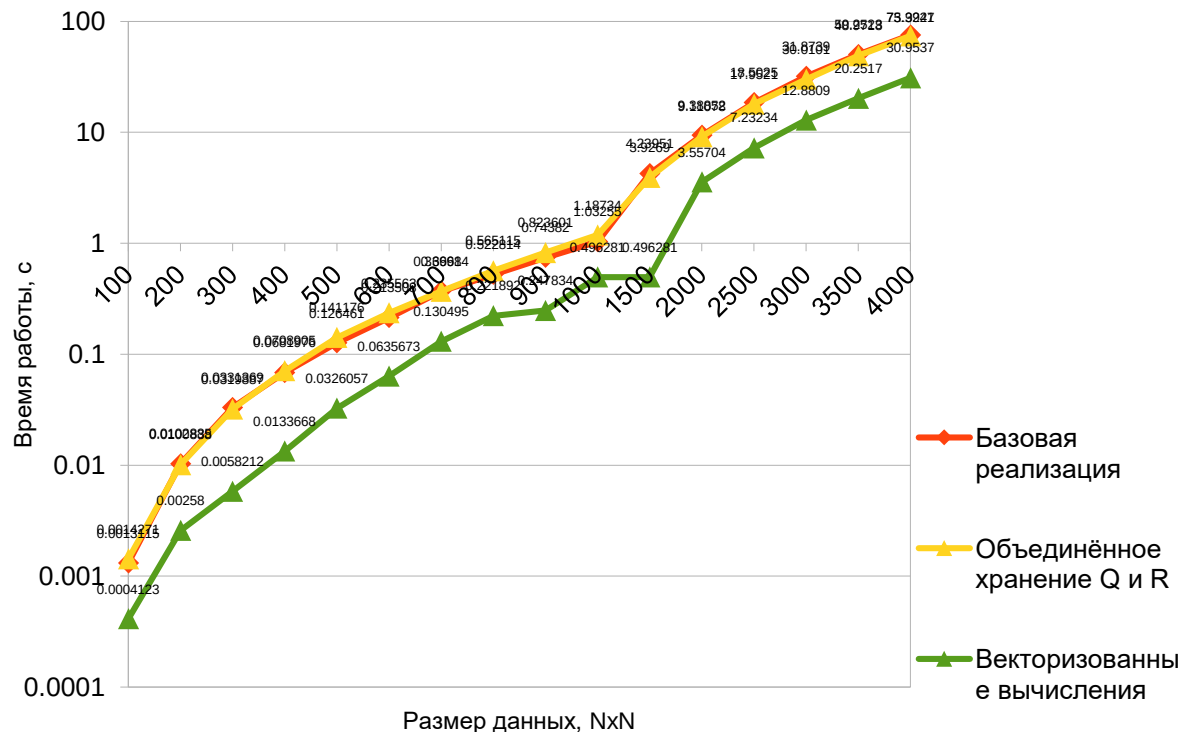
$$Q = \left(G_{n-1,n} G_{n-2,n} \dots G_{2n} G_{1n} \dots G_{24} G_{23} \dots G_{13} G_{12} \right)^T = G_{12}^T G_{13}^T \dots G_{1n}^T G_{23}^T G_{24}^T \dots G_{2n}^T \dots G_{n-2,n}^T G_{n-1,n}^T$$

Возможные оптимизации

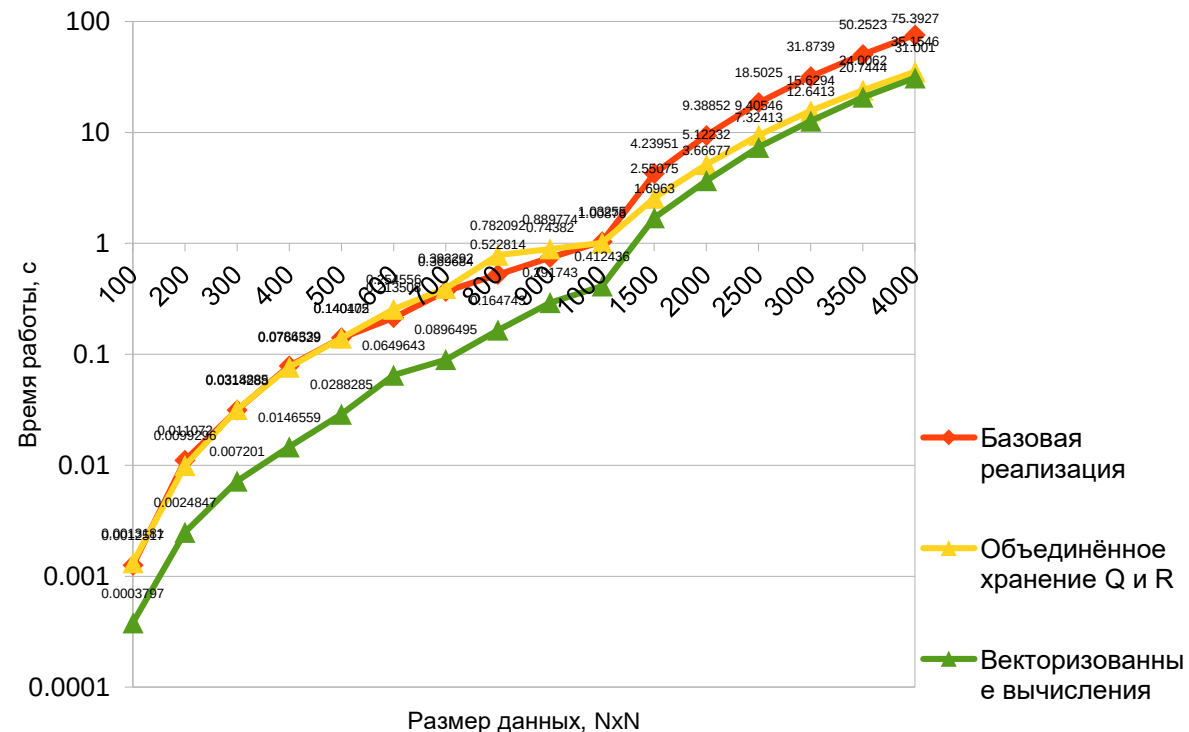
- Для эффективной работы с памятью матрицу Q нужно хранить транспонированной
- Сравнить два подхода вычисления Q и R : в рамках одного цикла и отдельно, сохраняя параметры вращения в отдельной структуре либо в матрице R
- Попробовать хранить Q и R в одной структуре для улучшения локальности кэша
- Можно не выполнять вращение строки при вычислении матрицы R целиком, поскольку слева от текущего столбца элементы вращаемых строк равны 0.
- Можно ещё эффективнее обращаться к памяти, выполняя вычисления сразу над несколькими последовательно лежащими элементами, применив векторизацию вычислений

Сравнение трёх полученных реализаций

С учётом оптимизаций, ниже представлены замеры однопоточного и параллельного исполнения для трёх программных реализаций:



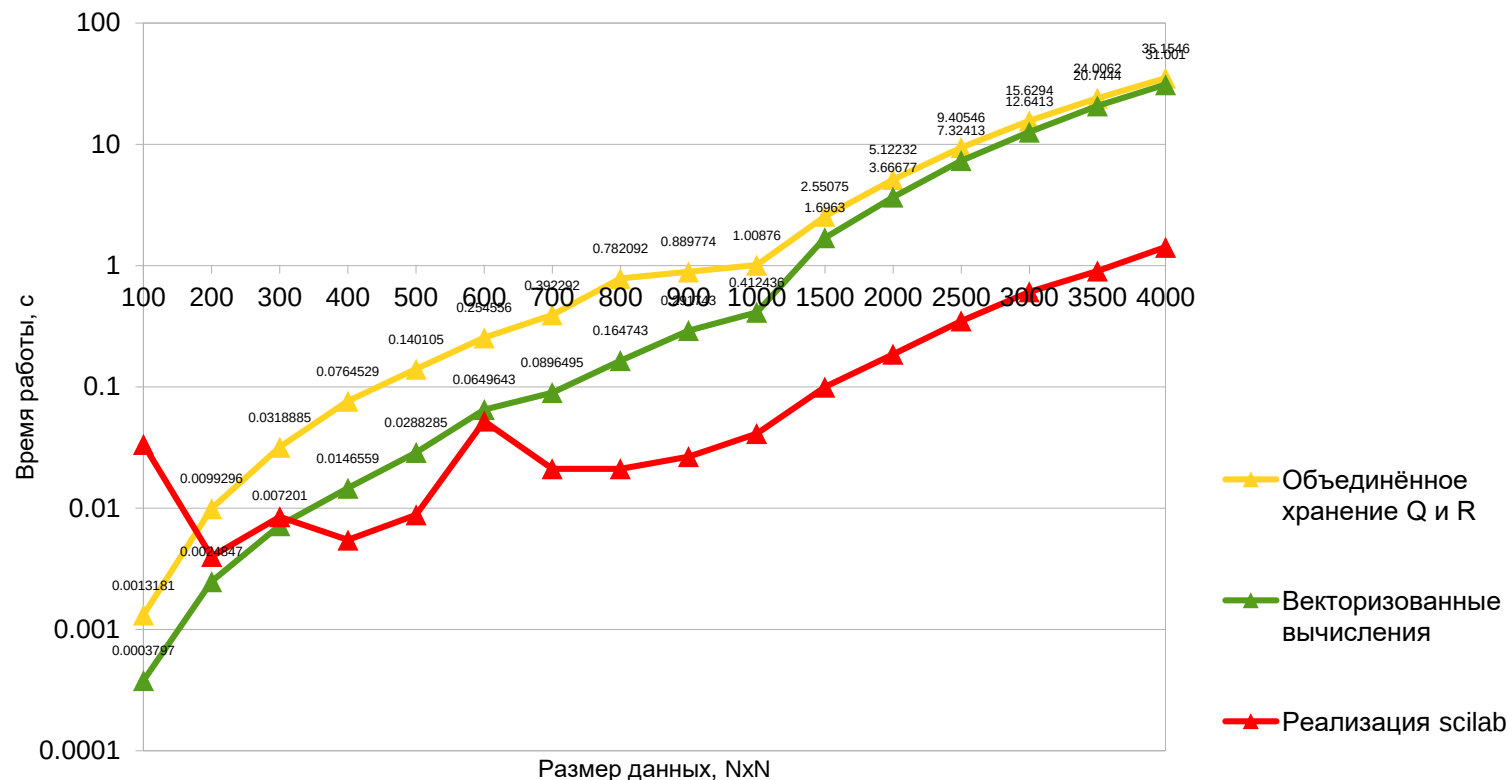
Запуск на одном потоке



Запуск на восьми потоках

Сравнение с библиотекой scilab

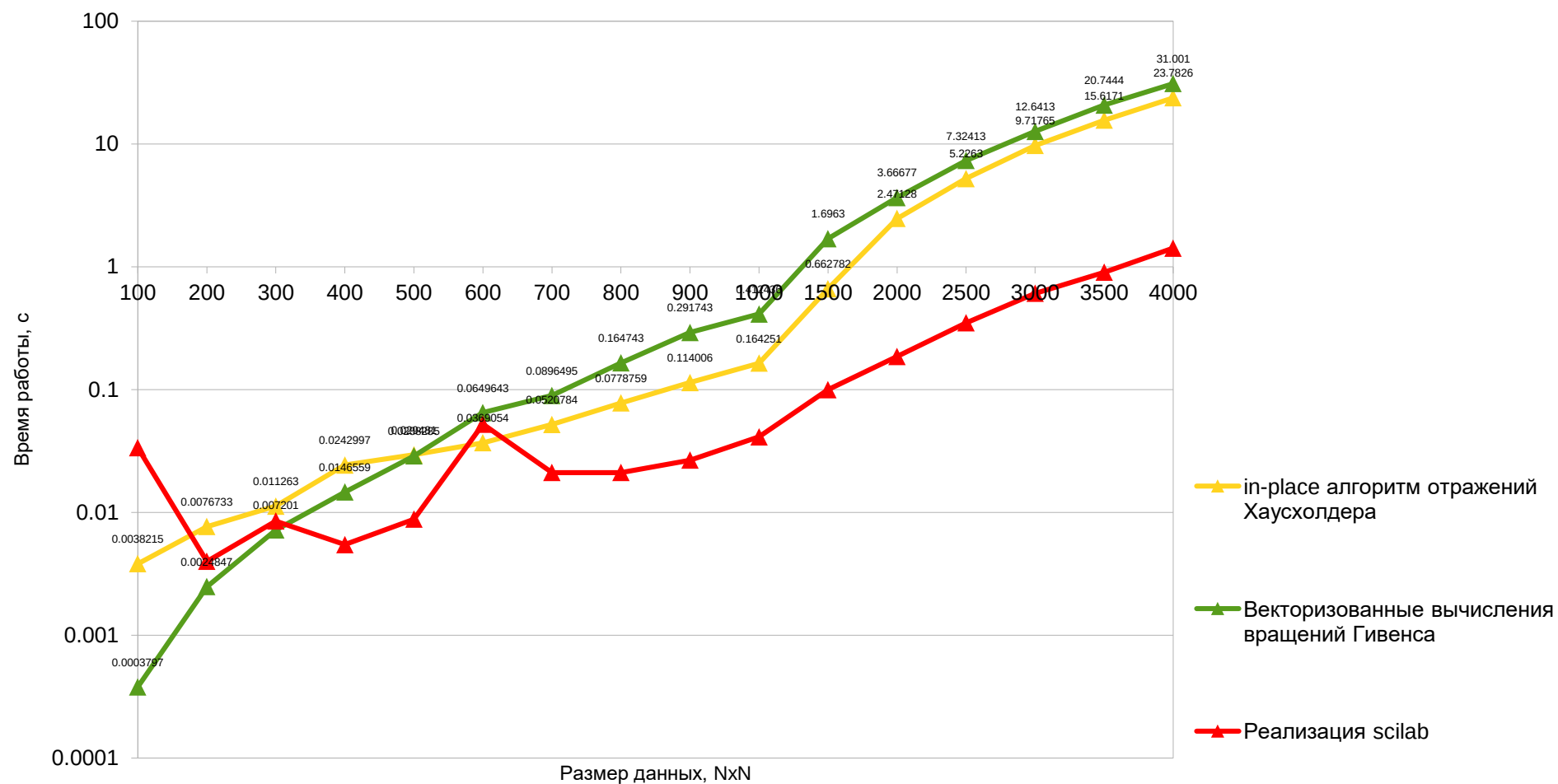
Ниже приведено сравнение работы двух оптимальных версий, исполняющихся на восьми потоках, с реализацией QR-разложения в библиотеке scilab:



- Scilab использует отражения Хаусхолдера для зануления целых столбцов, в то время как метод Гивенса лучше работает для разреженных матриц.
- Игнорирование нулевых отдельно стоящих значений позволит выполнить QR разложение быстрее.



Сравнение трёх реализаций



Матрица Хессенберга

Матрица Хессенберга — вид квадратной матрицы, обобщающий треугольные матрицы. Названа так в честь немецкого математика, Карла Хессенберга.

Верхняя матрица Хессенберга — это квадратная матрица, все элементы ниже побочной диагонали которой равны нулю:

$$H_{upper} = \begin{pmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2n} \\ 0 & h_{32} & h_{33} & \cdots & h_{3n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & h_{nn-1} & h_{nn} \end{pmatrix}$$

$$H_{low} = \begin{pmatrix} h_{11} & h_{12} & 0 & \cdots & 0 \\ h_{21} & h_{22} & h_{23} & \cdots & \vdots \\ h_{31} & h_{32} & h_{33} & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & h_{n-1n} \\ h_{n1} & h_{n2} & h_{n3} & \cdots & h_{nn} \end{pmatrix}$$

Аналогично определяется нижняя матрица Хессенберга, при транспонировании которой получается верхняя матрица Хессенберга.

QR-разложение верхней матрицы Хессенберга

Пусть верхняя хессенбергова матрица A имеет размер 6×6 . С помощью алгоритма QR-разложения на основе вращений Гивенса решение задачи разложения верхней Хессенберговой матрицы выглядит следующим образом:

$$H_{upper} = A = \begin{pmatrix} x & x & x & x & x & x \\ \textcolor{red}{x} & x & x & x & x & x \\ 0 & \textcolor{red}{x} & x & x & x & x \\ 0 & 0 & \textcolor{red}{x} & x & x & x \\ 0 & 0 & 0 & \textcolor{red}{x} & x & x \\ 0 & 0 & 0 & 0 & \textcolor{red}{x} & x \end{pmatrix} \quad G_{23} \cdot G_{12} \cdot A = \begin{pmatrix} x & x & x & x & x & x \\ \textcolor{red}{0} & x & x & x & x & x \\ 0 & \textcolor{red}{0} & x & x & x & x \\ 0 & 0 & \textcolor{blue}{x} & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & 0 & x & x \end{pmatrix} \quad G_{34} \cdot G_{23} \cdot G_{12} \cdot A = \begin{pmatrix} x & x & x & x & x & x \\ \textcolor{red}{0} & x & x & x & x & x \\ 0 & \textcolor{red}{0} & x & x & x & x \\ 0 & 0 & \textcolor{red}{0} & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & 0 & x & x \end{pmatrix}$$

необходимо с помощью вращений Гивенса последовательно занулить $(N-1)$ элементов, лежащих на побочной диагонали под главной (выделены красным)

QR-разложение нижней матрицы Хессенберга

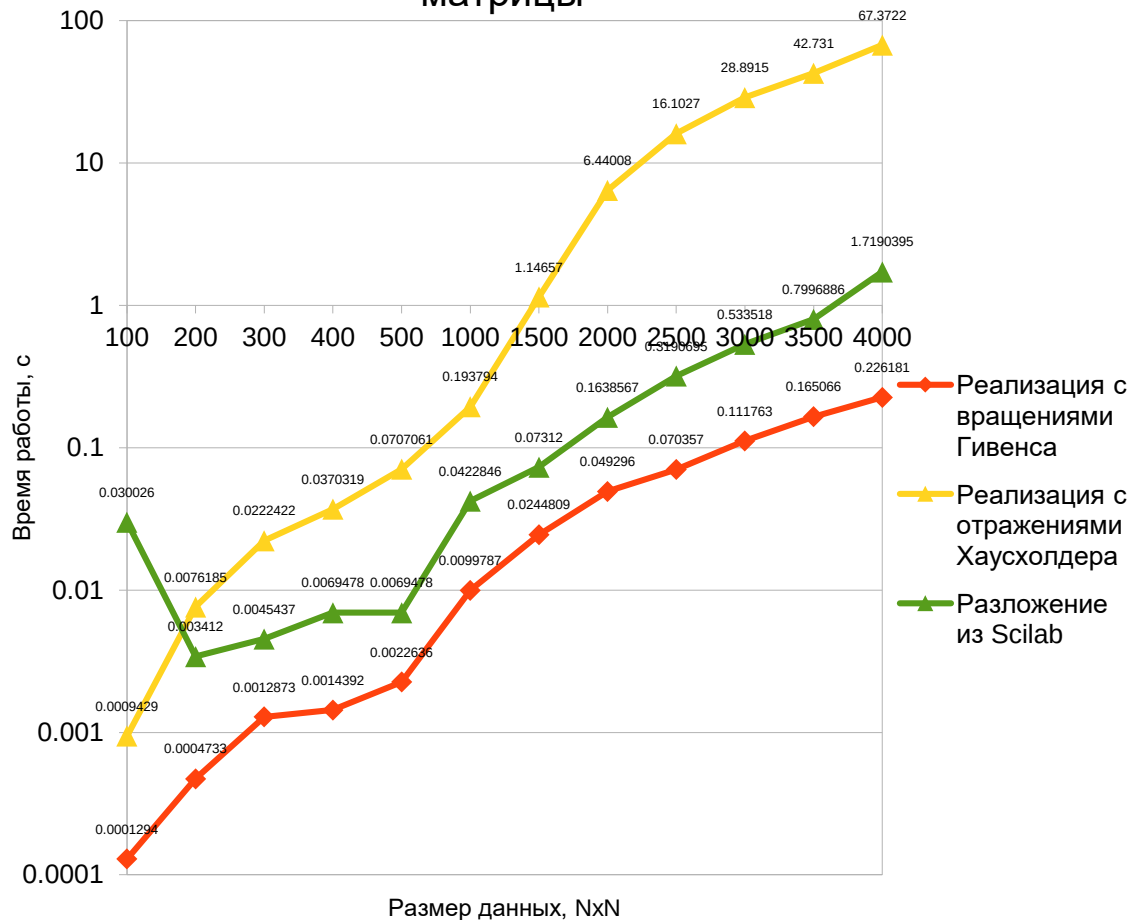
Пусть нижняя хессенбергова матрица A имеет размер 6×6 . С помощью алгоритма QR-разложения на основе отражений Хасхолдера решение задачи разложения нижней Хессенберговой матрицы выглядит следующим образом:

$$H_{low} = A = \begin{pmatrix} x & x & 0 & 0 & 0 & 0 \\ \color{red}{x} & x & x & 0 & 0 & 0 \\ \color{red}{x} & \color{red}{x} & x & x & 0 & 0 \\ \color{red}{x} & \color{red}{x} & \color{red}{x} & x & x & 0 \\ \color{red}{x} & \color{red}{x} & \color{red}{x} & \color{red}{x} & x & x \\ \color{red}{x} & \color{red}{x} & \color{red}{x} & \color{red}{x} & \color{red}{x} & x \end{pmatrix} \quad H_2 \cdot H_1 \cdot A = \begin{pmatrix} x & x & 0 & 0 & 0 & 0 \\ \color{red}{0} & x & x & 0 & 0 & 0 \\ \color{red}{0} & \color{red}{0} & x & x & 0 & 0 \\ \color{red}{0} & \color{red}{0} & \color{blue}{x} & x & x & 0 \\ \color{red}{0} & \color{red}{0} & \color{blue}{x} & x & x & x \\ \color{red}{0} & \color{red}{0} & \color{blue}{x} & x & x & x \end{pmatrix} \quad H_3 \cdot H_2 \cdot H_1 \cdot A = \begin{pmatrix} x & x & x & 0 & 0 & 0 \\ \color{red}{0} & x & x & 0 & 0 & 0 \\ \color{red}{0} & \color{red}{0} & x & x & 0 & 0 \\ \color{red}{0} & \color{red}{0} & \color{red}{0} & x & x & 0 \\ \color{red}{0} & \color{red}{0} & \color{red}{0} & x & x & x \\ \color{red}{0} & \color{red}{0} & \color{red}{0} & x & x & x \end{pmatrix}$$

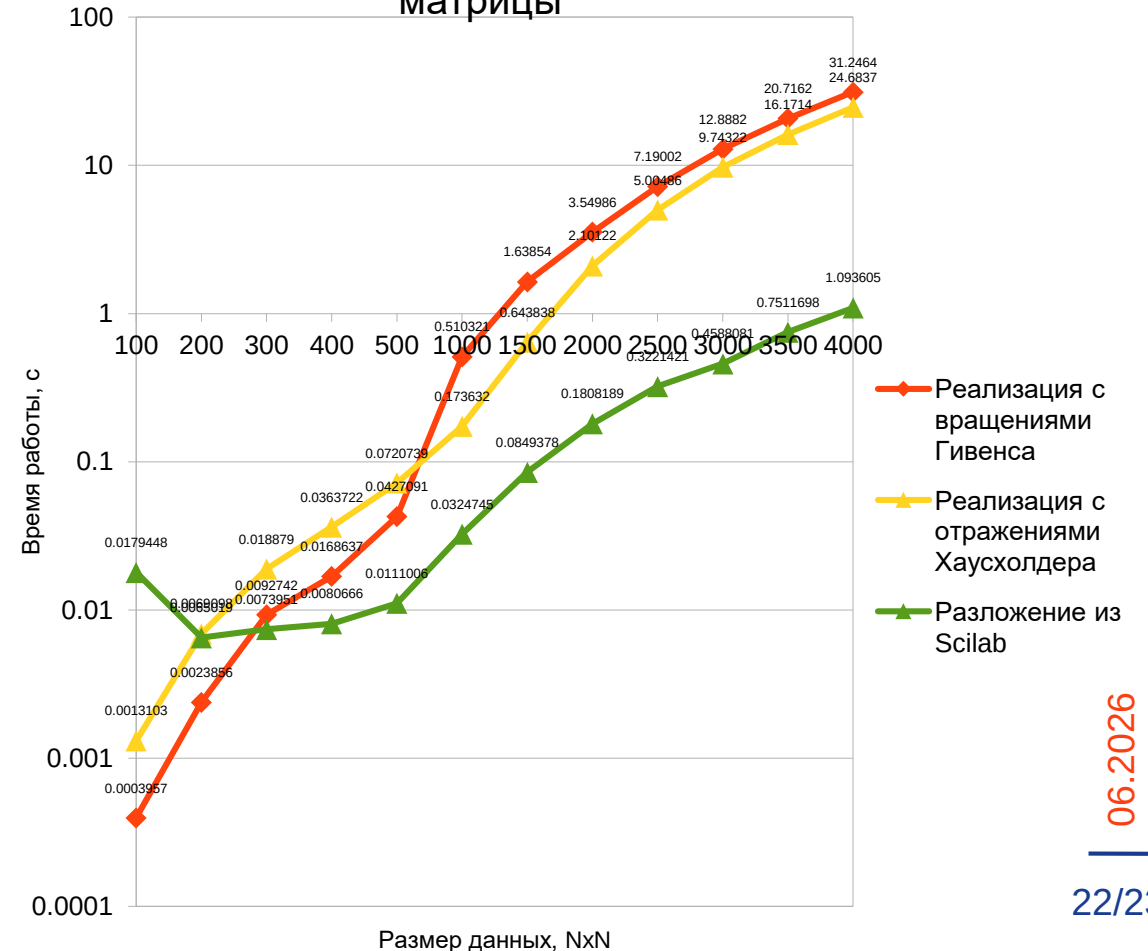
необходимо с помощью отражений Хаусхолдераа занулить $\left(\frac{N^2 - N}{2}\right)$ элементов, лежащих ниже главной диагонали (выделены красным)

Результаты решения задачи QR-разложения хессенберговой матрицы

Сравнение скорости решения QR-разложения верхней хессенберговой матрицы



Сравнение скорости решения QR-разложения нижней хессенберговой матрицы



Заключение

- QR-разложение на основе отражений Хаусхолдера удалось лучше оптимизировать в виду большего пространства для улучшений: избавление от матричных умножений и in-place хранение векторов отражения дали очень большой прирост производительности.
- QR-разложение на основе вращений Гивенса в целом работает быстрее на небольших данных (<1000), однако всё ещё проигрывает по скорости на больших ($>> 1000$). Дело в ограниченной применимости метода — этот метод даст лучший результат при применении его к разреженным матрицам.
- В связи с этим целесообразно перед применением метода проанализировать степень заполненности поддиагональной части матрицы 0. Эксперименты показали, что метод имеет смысл применять при $>90\%$ процентов нулей.
- Также можно попробовать реализовать блочный алгоритм. Для этого больше подойдёт MPI-параллелизм на уровне процессов. Это позволит приблизиться к версии из библиотеки Scilab, где QR-разложение реализуется с использованием подпрограмм из библиотеки Lapack.



СПАСИБО ЗА ВНИМАНИЕ!